

Quadrotor Control and Evaluation Simulation on NVIDIA Jetson

Ayberk Aygün* and Berkay Taşkın†

*Computer Engineering Department, Hacettepe University, Ankara, Türkiye

†Graduate School of Science and Engineering, Hacettepe University, Ankara, Türkiye

E-mails: *ayberk.aygun07@gmail.com, †berkaytaskinw@gmail.com

Abstract—Unmanned aerial vehicles rely heavily on control algorithms to maintain their stability and execute their missions. Evaluating controller robustness under uncertainties such as inertia variations, additive control noise, and varying initial states typically requires Monte Carlo simulation, which becomes computationally expensive when performed sequentially on a CPU. In this project, we adapt the GPU Accelerated Trajectory Evaluation (GATE) framework for Monte Carlo-based controller evaluation on an NVIDIA Jetson embedded platform. A cascaded PID controller drives a nonlinear quadrotor simulation on the CPU, while GATE’s CUDA kernel structure runs parallel trajectory rollouts on the GPU under randomized perturbations in moment of inertia and control input noise. The GPU-side evaluation also feeds back into a PID gain selector that compares candidate gain sets across the Monte Carlo results and selects the best-performing pair to update the controller. The evaluation covers CPU-only versus CPU-plus-GPU execution time, scalability with increasing rollout count, and real-time feasibility of the control loop on the Jetson’s ARM cores.

Index Terms—Quadrotor, PID, Controller Evaluation, Monte Carlo, Simulation.

I. INTRODUCTION

Unmanned aerial vehicles (UAVs) have become increasingly prevalent across a wide range of applications including surveillance, delivery, search and rescue, and agricultural monitoring. The reliable operation of these vehicles depends heavily on the performance of their control algorithms, which must maintain stability and track desired trajectories under real-world conditions. In practice, drone controllers face numerous challenges such as sensor noise, external disturbances like wind gusts, model uncertainties in physical parameters, and variations in initial flight conditions. Designing a single controller that performs robustly across all these uncertainty sources remains a significant challenge in the field of UAV control.

A widely adopted approach for evaluating controller robustness is Monte Carlo simulation, where the controller is tested repeatedly under randomized scenarios by injecting perturbations drawn from known probability distributions and observing the resulting system behavior [1]. While this method provides statistically meaningful robustness metrics, it requires running a large number of independent trajectory simulations. When these simulations are executed sequentially on a CPU, the computational cost becomes prohibitive, limiting both the number of test scenarios that can be analyzed and the speed at which evaluation results become available. This limitation

is particularly critical on embedded platforms where computational resources are already constrained.

Graphics processing units (GPUs) have been demonstrated as an effective platform for accelerating Monte Carlo simulations through parallelization. In the context of controller evaluation, the GPU Accelerated Trajectory Evaluation (GATE) framework [2] provides a C++/CUDA-based tool that distributes independent trajectory rollouts across GPU threads. GATE separates the simulation into three modular components — dynamics, guidance, and perturbations — each managing its own GPU memory through a managed base class. The framework employs template-based static polymorphism (CRTP) to avoid virtual function table overhead on the GPU, and provides a plug-and-play architecture where different controllers and dynamic models can be integrated without modifying the GPU backend.

Separately, prior work on real-time quadrotor simulation has addressed the challenge of running nonlinear quadrotor dynamics within tight execution time budgets by proposing computational complexity-based approaches [3]. These methods provide a foundation for implementing physically accurate simulations on platforms where timing constraints must be respected, which is directly relevant to embedded deployment scenarios.

In this project, we combine these two lines of work by adapting GATE’s GPU-parallel Monte Carlo architecture [2] for use with a cascaded PID controller on an NVIDIA Jetson embedded platform. The Jetson board provides both multi-core ARM CPUs and an integrated CUDA-capable GPU within a single board, making it a suitable platform for heterogeneous workload partitioning. The quadrotor is modeled as a nonlinear system whose state comprises position, velocity, Euler attitude angles, and body-frame angular rates. A cascaded PID controller is designed following the standard approach used in real-world flight controllers: the outer loop converts position error into desired attitude references, and the inner loop tracks these references by commanding motor torques through a motor mixer. Controller gains are tuned offline through closed-loop pole placement analysis to ensure numerical stability with the forward Euler integrator at a 5 ms step size.

The system partitions the workload between CPU and GPU following GATE’s design philosophy [2]. The CPU executes the nominal real-time control loop, while the GPU handles Monte Carlo evaluation under perturbations in moment of

inertia and additive control noise. The GPU-side results also feed into a PID gain selector module that compares candidate gain sets across the Monte Carlo scenarios and updates the controller with the best-performing pair. This closes the loop between robustness evaluation and controller tuning within the embedded platform. Unlike [1], which applies Monte Carlo methods within an MPC framework, our approach focuses on evaluating and adapting a cascaded PID controller on embedded hardware.

The main contributions of this work are: a proof-of-concept demonstration that GATE's GPU-parallel Monte Carlo architecture can be effectively deployed on a resource-constrained embedded platform; quantitative CPU-only versus CPU-plus-GPU performance comparisons and scalability analysis on the Jetson hardware; and a reusable modular framework where the PID controller, dynamics model, or perturbation definitions can be independently replaced for future work on scenario-based evaluation and simulation-to-real experimentation.

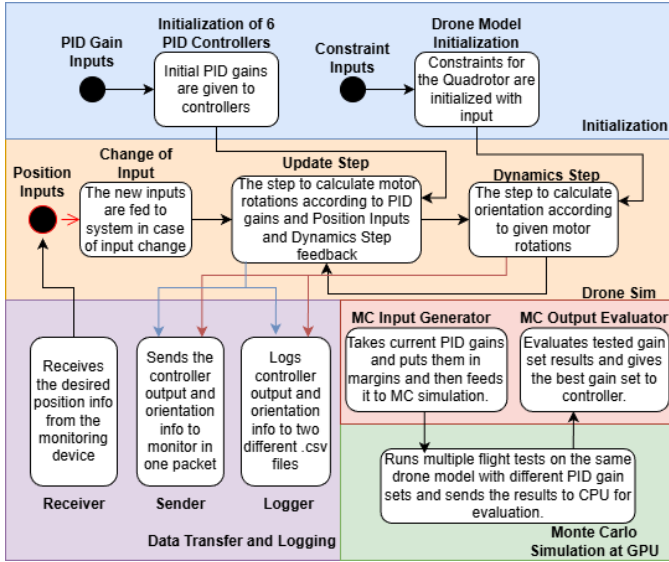


Fig. 1. A simplified representation of the system

II. RELATED WORK

Our project has several key characteristics. It is a quadrotor dynamic model running on the Jetson CPU producing a 6-DoF Quadrotor simulation. It has six decoupled PID controllers that govern the quadrotor orientation. GPU-accelerated Monte Carlo simulation on the Jetson GPU to determine the best set of PID gains in parallel, and finally an embedded architecture where both simulation and optimization reside on a single Jetson board, enabling potential on board real-time tuning. We have found some work relating to some of these aspects. The work Designing a Self-Tuning PID Controller for AR Drone Quadrotor [4] uses a PID as a controller for the quadrotor and tunes the PID controller with a gradient-descent algorithm during flight. The similarity of this project comes from the idea of tuning quadrotor controller during flight but the difference is the tuning method where our work performs tuning with GPU

accelerated Monte Carlo simulation while this work performs it with a gradient-descent algorithm. Gradient-descent algorithm may execute faster but it may converge to a local optima, it is sensitive to learning rate and it does not explore the full parameter space like our Monte Carlo simulation method.

Motor-Quadrotor Simulations Based on FOC and Cuckoo PID Tuning [5] is another PID controller based quadrotor simulation work where the parameters are tuned with Cuckoo Optimization Algorithm (COA) minimizes the squared-error cost functions considering three phases. Quadrotor motion dynamics, motor speed dynamics, and motor current dynamics. This work presents a more detailed drone simulation in comparison to ours where it works as a PC simulation as it has a lot more resources to use. This work is not designed for an embedded computer and be computationally expensive for it, where one of our key concepts is optimizing on the flight.

Modelling, Simulation, and Implementation of PID Controller on Quadrotors [6] work uses a drone dynamical model similar to ours and designs a PID controller for it. The work contains both simulation and implementation of the algorithm which is also in line with our aim. Where in this work the PID design is done manually and proceeds to the test the algorithm in simulation. This can be time consuming as there are other automated optimization methods such as the one we present.

A High-Performance Monte Carlo Simulation Toolbox for Uncertainty Quantification of Closed-Loop Systems [7] presents a parallelized Monte Carlo simulation toolbox implemented in C for shared-memory architectures. The core idea of this paper is the same, use massively parallel Monte Carlo simulation for controller (PID) tuning and performance quantification. It also demonstrates automatic PID tuning through Monte Carlo search which is another goal that we have. However, the application domain is a bioreactor not a quadrotor and this project runs on a high-performance workstation not on an embedded platform. Also, this work presents a CPU-only parallelism where we try to do it on GPU architecture.

Dynamic Response Optimization for Quadrotor UAVs Using a Parameter Self-Tuning Fuzzy PID Control Method [8] also focuses on PID gain adjustment where it presents a light-weight fuzzy logic rule set for PID gain optimization. It also addresses limitations of fixed PID gains and uses a simulation based evaluation. However, even though the fuzzy logic algorithm is light-weight and suitable for on board deployment, the performance depends heavily on the quality of hand-crafted fuzzy rules and the system does not explore the gain space for optimality.

Intelligent Controller Design for Quad-Rotor Stabilization in Presence of Parameter Variations [9] is another work that present self-tuning PID controller for quadrotors and its strengths and weaknesses are similar. This work is validated on a well-known physics simulator Gazebo and on real hardware with several experiments. Its success relies heavily on hand crafted fuzzy rules which requires a lot of expertise on the system and it is still not guaranteed to be suitable for many applications.

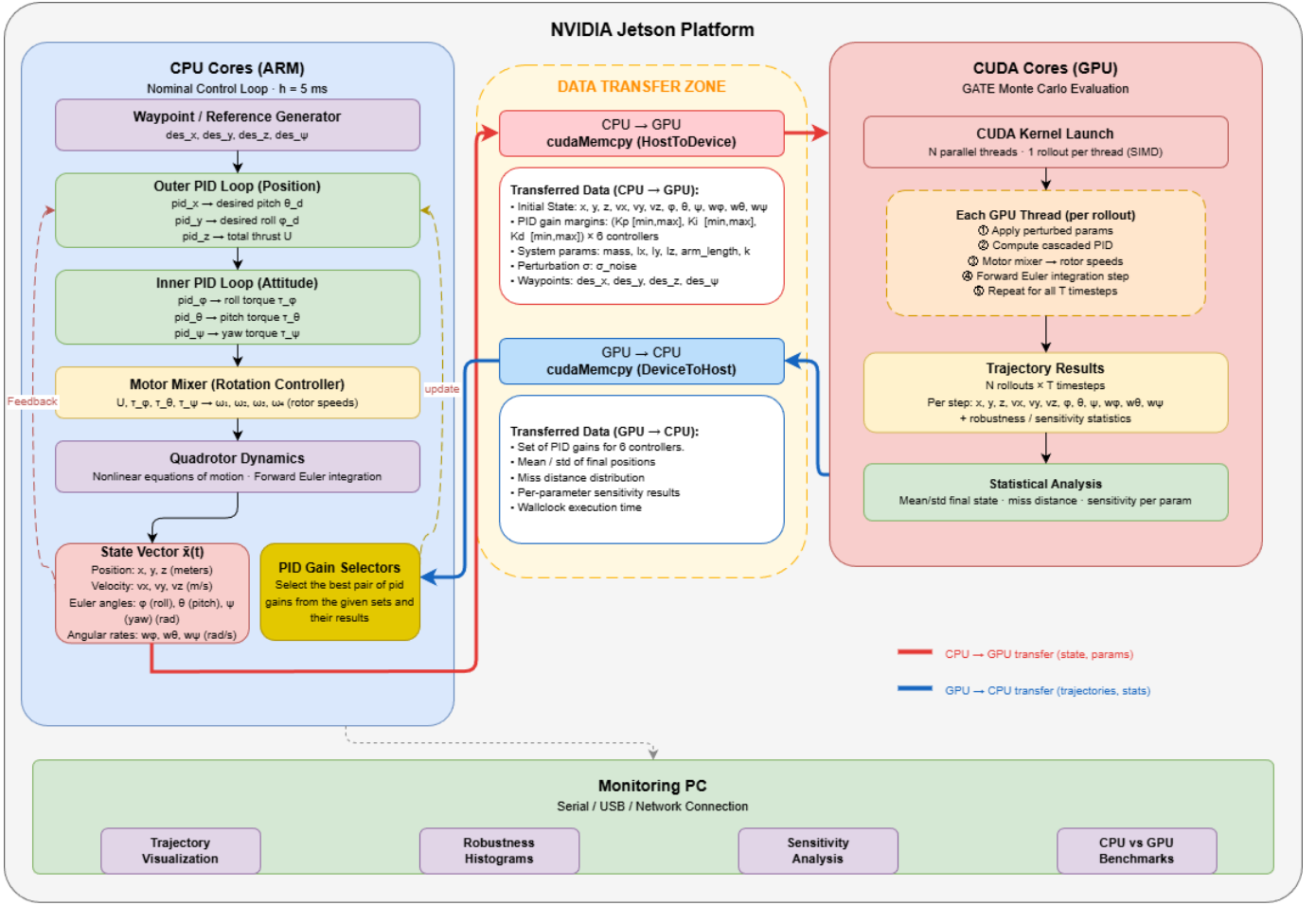


Fig. 2. More detailed representation of the system

Fast Monte Carlo Analysis for 6-DoF Powered-Descent Guidance via GPU-Accelerated Sequential Convex Programming [10] introduces a GPU-accelerated Monte Carlo framework for a powered-descent trajectory optimization for rockets. This paper presents a heavy algorithm that needs a high-performance desktop/server GPU's and the controller structure is different from ours. However, even though the application field and the hardware is different, the core methodology of GPU-accelerated Monte Carlo simulation is the same. The implementation presented in this paper can be a guidance and inspiration for our work while we implement our own methodologies.

III. PROBLEM DEFINITION

This section formally defines the quadrotor simulation and controller evaluation problem, including the system description, objectives, and constraints for the embedded heterogeneous platform.

A. System / Application Description

The application scenario consists of a quadrotor UAV simulation running on an NVIDIA Jetson embedded platform, where the CPU handles the nominal flight control loop and

TABLE I

Work	Year	PID Controller	Tuning Automation	Embedded Device	Validation with Simulation	GPU Usage
Self-Tuning PID AR	2017	✓	✓	✓	✓	
Drone [4]						
FOC + Cuckoo PID [5]	2024	✓	✓		✓	
Modelling and PID on Quad [6]	2021	✓		✓	✓	
MC Toolbox Closed-Loop [7]	2022	✓	✓		✓	
Fuzzy PID Quad UAV [8]	2024	✓	✓		✓	
Intelligent Ctrl Quad [9]	2017	✓	✓	✓	✓	
CPU MC 6-DoF Descent [10]	2024		✓		✓	✓
This Work	2026	✓	✓	✓	✓	✓

the GPU accelerates Monte Carlo-based controller robustness evaluation using the GATE framework architecture.

The quadrotor is modeled as a rigid body with four rotors generating thrust and torque in the body frame. The system state is defined as:

$$\bar{x} = [x, y, z, v_x, v_y, v_z, \phi, \theta, \psi, w_\phi, w_\theta, w_\psi]^T \quad (1)$$

where x, y, z denote the position of the center of mass in the

inertial frame, v_x, v_y, v_z are the corresponding translational velocities, ϕ (roll), θ (pitch), and ψ (yaw) are the Euler attitude angles, and w_ϕ, w_θ, w_ψ are the angular velocities about the body-frame axes.

Each rotor i generates a force proportional to the square of its angular velocity:

$$f_i = k \cdot \omega_i^2, \quad i \in \{1, 2, 3, 4\} \quad (2)$$

where k is the propeller constant. The total thrust and torques are computed from the individual rotor forces as:

$$U = f_1 + f_2 + f_3 + f_4 \quad (3)$$

$$\tau_\psi = (f_1 + f_3 - f_2 - f_4) \cdot L \quad (4)$$

$$\tau_\theta = (f_1 - f_3) \cdot L \quad (5)$$

$$\tau_\phi = (f_2 - f_4) \cdot L \quad (6)$$

where L is the arm length from the center of mass to each rotor.

The translational accelerations in the inertial frame are:

$$a_x = \frac{U}{m} (\sin \psi \sin \phi + \cos \psi \sin \theta \cos \phi) \quad (7)$$

$$a_y = \frac{U}{m} (-\cos \psi \sin \phi + \sin \psi \sin \theta \cos \phi) \quad (8)$$

$$a_z = \frac{U}{m} (\cos \theta \cos \phi) - g \quad (9)$$

where m is the quadrotor mass and $g = 9.8 \text{ m/s}^2$ is gravitational acceleration. The angular accelerations are governed by the Euler rotational equations:

$$\ddot{\psi} = \frac{(I_x + I_z - I_y) \cdot w_\phi \cdot w_\theta + \tau_\psi}{I_z} \quad (10)$$

$$\ddot{\theta} = \frac{(I_z - I_x - I_y) \cdot w_\psi \cdot w_\phi + \tau_\theta}{I_y} \quad (11)$$

$$\ddot{\phi} = \frac{(I_x + I_y - I_z) \cdot w_\psi \cdot w_\theta + \tau_\phi}{I_x} \quad (12)$$

where I_x, I_y, I_z are the principal moments of inertia. The state is integrated forward in time using the forward Euler method with a fixed step size $h = 0.005 \text{ s}$:

$$\bar{x}(t+h) = \bar{x}(t) + h \cdot \dot{\bar{x}}(t) \quad (13)$$

The nominal system parameters used in this work are: $m = 1.4 \text{ kg}$, $I_x = I_y = 0.0116 \text{ kg}\cdot\text{m}^2$, $I_z = 0.0232 \text{ kg}\cdot\text{m}^2$, $L = 1.0 \text{ m}$, and $k = 10^{-6}$.

The controller follows a cascaded PID structure with an outer position loop and an inner attitude loop, reflecting the standard architecture used in real-world flight controllers. The outer position loop uses three PID controllers to generate the desired attitude references and total thrust from position error:

$$U = m \cdot g + \text{PID}_z(z_{des} - z) \quad (14)$$

$$\theta_d = \text{PID}_x(x_{des} - x) \quad (15)$$

$$\phi_d = -\text{PID}_y(y_{des} - y) \quad (16)$$

The desired angles are clamped to $\pm 0.44 \text{ rad}$ ($\approx 25^\circ$) to maintain validity of the small-angle approximation. The inner

attitude loop uses three PID controllers to generate torque commands from attitude error:

$$\tau_\phi = \text{PID}_\phi(\phi_d - \phi) \quad (17)$$

$$\tau_\theta = \text{PID}_\theta(\theta_d - \theta) \quad (18)$$

$$\tau_\psi = \text{PID}_\psi(\psi_{des} - \psi) \quad (19)$$

Each PID controller implements derivative-on-measurement and anti-windup:

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau - K_d \frac{d(\text{measured})}{dt} \quad (20)$$

where the derivative term acts on the measured value rather than the error to avoid derivative kick on setpoint changes, and the integral term is clamped to a configurable limit to prevent windup. Controller gains are selected based on target natural frequencies to ensure all closed-loop poles satisfy the Euler stability criterion $|1 + h \cdot s| < 1$. The inner attitude loop targets $\omega_n = 40 \text{ rad/s}$ and the outer position loop targets $\omega_n = 1.5 \text{ rad/s}$, providing a bandwidth separation ratio of approximately 20:1 for cascade stability.

The motor mixer maps the total thrust and torques to individual rotor forces:

$$f_1 = \frac{U}{4} + \frac{\tau_\psi}{4L} + \frac{\tau_\theta}{2L} \quad (21)$$

$$f_2 = \frac{U}{4} - \frac{\tau_\psi}{4L} + \frac{\tau_\phi}{2L} \quad (22)$$

$$f_3 = \frac{U}{4} + \frac{\tau_\psi}{4L} - \frac{\tau_\theta}{2L} \quad (23)$$

$$f_4 = \frac{U}{4} - \frac{\tau_\psi}{4L} - \frac{\tau_\phi}{2L} \quad (24)$$

Each force is clamped to zero from below (motor saturation), and rotor speeds are obtained as $\omega_i = \sqrt{f_i/k}$.

For Monte Carlo evaluation, N independent rollouts are simulated on the GPU. The control noise $w_i(t)$ is added to the motor forces at each timestep. Each rollout i produces a full state trajectory $\bar{x}_i(t)$ for $t \in [0, T]$ using the same dynamics equations but different PID controllers. For each candidate gain set $K_j = \{K_{p,j}, K_{i,j}, K_{d,j}\}$, the GPU evaluates N rollouts and computes performance metrics. The gain selector module compares these results and selects the gain set that minimizes the chosen cost metric, then transfers the selected gains back to the CPU to update the outer PID loop.

The key assumptions of the system are: the quadrotor operates in a simulation environment without real sensor hardware; low-level aerodynamic effects such as blade flapping and ground effect are neglected; perturbations are drawn from Gaussian distributions with known standard deviations; and the forward Euler integrator is sufficient for the dynamics at the chosen step size.

B. Objectives

The objectives of this project are as follows:

- Demonstrate that the GATE GPU-parallel Monte Carlo architecture can be deployed on a resource-constrained

NVIDIA Jetson embedded platform for controller robustness evaluation.

- Maintain real-time execution of the CPU-side cascaded PID control loop within the 5 ms step size on the Jetson's ARM cores.
- Maximize GPU-side Monte Carlo evaluation throughput by running N parallel trajectory rollouts simultaneously using CUDA.
- Enable PID gain selection by evaluating multiple candidate gain sets across Monte Carlo scenarios on the GPU and feeding the best-performing set back to the CPU controller.
- Provide quantitative performance comparisons between CPU-only sequential evaluation and CPU+GPU parallel evaluation, measuring speedup ratio and scalability with increasing rollout count.
- Measure and characterize the CPU-GPU data transfer overhead to understand its impact on total evaluation latency.

C. Constraints

The system operates under the following constraints:

- **Timing:** The CPU control loop must complete one full iteration (state read, six PID computations, motor mixer, Euler integration) within 5 ms. Violation of this deadline results in simulation timing inaccuracy.
- **Hardware:** The system runs on a single NVIDIA Jetson board with shared memory between the ARM CPU and the integrated GPU. Both processors share the same power and thermal envelope.
- **Memory:** GPU device memory must accommodate N rollouts of perturbation data ($N \times T$ values per perturbation source) and $N \times T \times 12$ state trajectory output values simultaneously. The Jetson's GPU memory is significantly smaller than desktop-class GPUs, which limits the maximum N and T .
- **Data transfer:** All perturbation data and system parameters must be transferred from host (CPU) memory to device (GPU) memory via `cudaMemcpy` before kernel launch. Result trajectories must be transferred back after kernel completion. This transfer latency is part of the total evaluation time.
- **Numerical stability:** The forward Euler integrator can result in exponentially growing results where eventually all calculations converge to NaN. This constrains the maximum PID gains that can be used without causing divergence.
- **Software:** The system runs a minimal Linux image with CUDA toolkit on the Jetson. No real-time operating system (RTOS) is used; timing guarantees are best-effort under standard Linux scheduling.

IV. SYSTEM ARCHITECTURE AND DESIGN

A. Hardware/Software Architecture

The hardware will be a NVIDIA Jetson Nano development board. The programmable units of it are Integrated 128-core

Maxwell GPU and Quad-core ARM Cortex-A57 processor clocked at 1.43 GHz [11].

The schematic representation of the interaction between these two computational units is given in Figure 2. The system will run drone simulation, communicating with monitor tasks on CPU. Where the drone dynamics and controller algorithm runs at a specific time step. The GPU will run Monte Carlo simulations to test the performance of different controllers on a quadrotor model with constant parameters and give the results to CPU. The CPU is also responsible for starting the Monte Carlo simulation on GPU via feeding it the data it requires to operate and then update its controller from the data acquired from GPU when the Monte Carlo test is finished.

B. Dataflow and System Model

As shown in the Figure 1 the system steps are initialization, drone simulation, Monte Carlo simulation start, Monte Carlo simulation end and Data transfer and logging. The initialization part is self explanatory where the initial PID gains and system constraints such as time step or mechanical constraints defining the quadrotor are fed into the model. Drone simulation part runs periodically and responds to the desired position inputs coming from the monitoring device. The drone simulation part consists of two main parts which are update step and dynamics step.

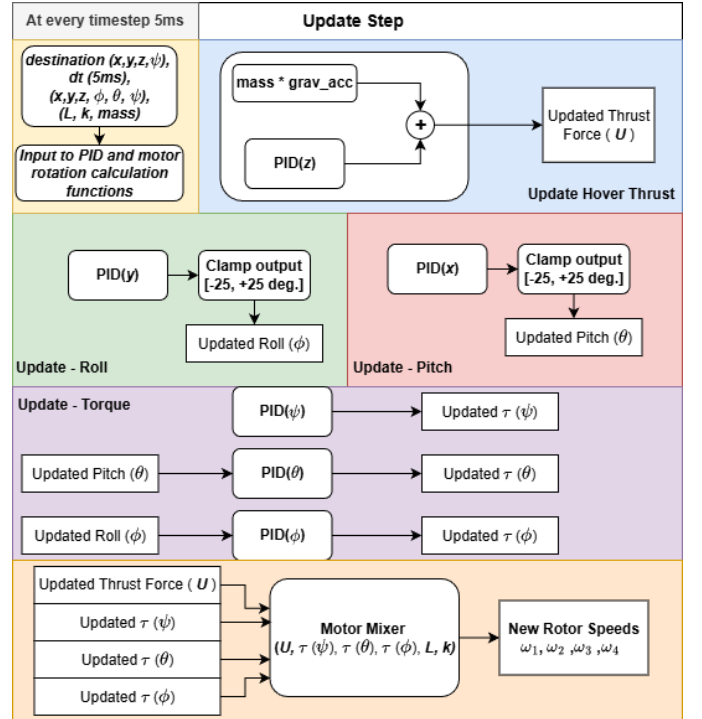


Fig. 3. Schematic of Update Step

Update step part is responsible for calculating the PID controller outputs (which are the new rotor speeds) and feed them to the dynamics part. Illustration of this is given in Figure 3.

Update step part runs 6 PID controller calculations in order to generate the required rotor speed outputs. The algorithm of these PID controllers are the same where the only differences between them are the PID gains. The schematic representing the flow is given in Figure 4.

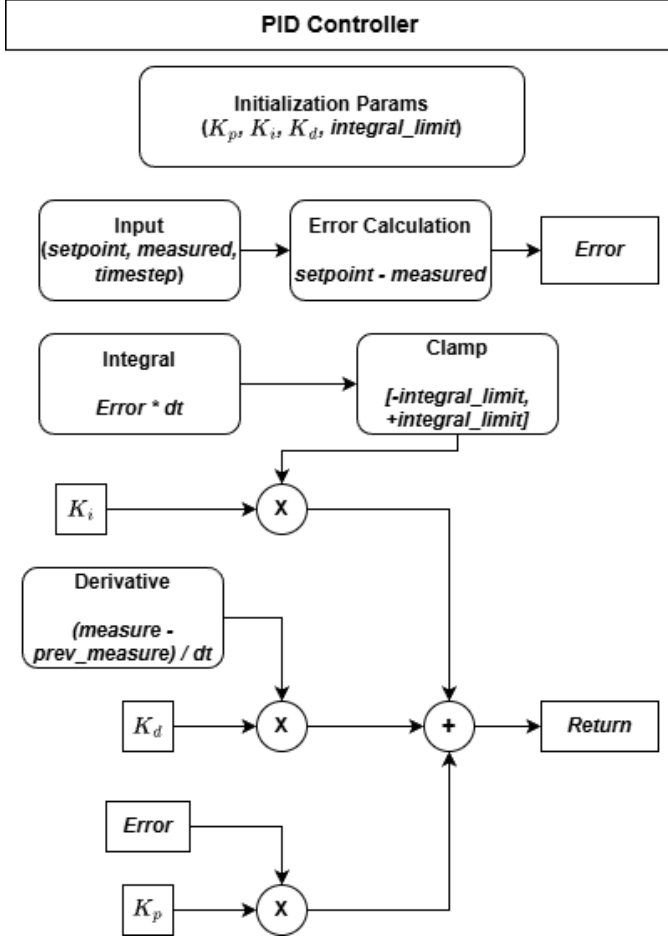


Fig. 4. PID Calculation Schematic

The Update Step also include a Motor Mixer part where in this the required rotations for each rotor are calculated separately. The schematic representing the calculation is given in Figure 5.

The Dynamics Step has three phases, calculation of angular and directional accelerations at time t, calculation of the velocities at time t + 1 according to accelerations and calculation of the orientation at time t + 1 according to velocities at time t. The acceleration calculation is done by implementing our drone dynamics model where the required values are initialized as zero, velocity and orientation part is done by Euler's numerical method. The Euler's numerical method is defined as following.

Given the initial value problem:

$$f(x, y) = \frac{dy}{dx} =, \quad y(x_0) = y_0 \quad (25)$$

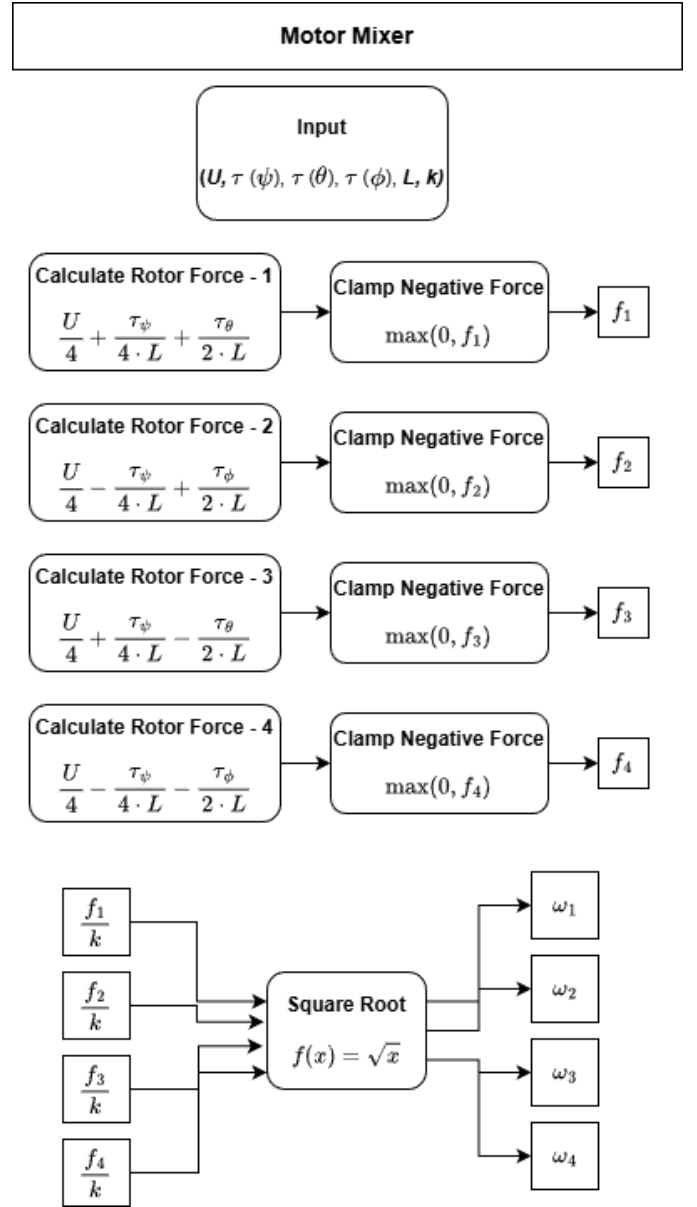


Fig. 5. Motor Mixer Schematic

The iterative update formula is:

$$y_{n+1} = y_n + h \cdot f(x_n, y_n) \quad (26)$$

where:

$$x_{n+1} = x_n + h \quad (27)$$

$$h = \text{step size} \quad (28)$$

$$n = 0, 1, 2, \dots \quad (29)$$

The dynamics calculations follow this logic and the implementation of them are illustrated in the figures 6 7 8.

C. Requirements Analysis (Implementation Track)

1) *Functional Requirements:* FR-01 — Quadrotor Dynamic Model Simulation The system shall simulate a 6-DoF quadrotor dynamic model on the Jetson CPU, computing translational

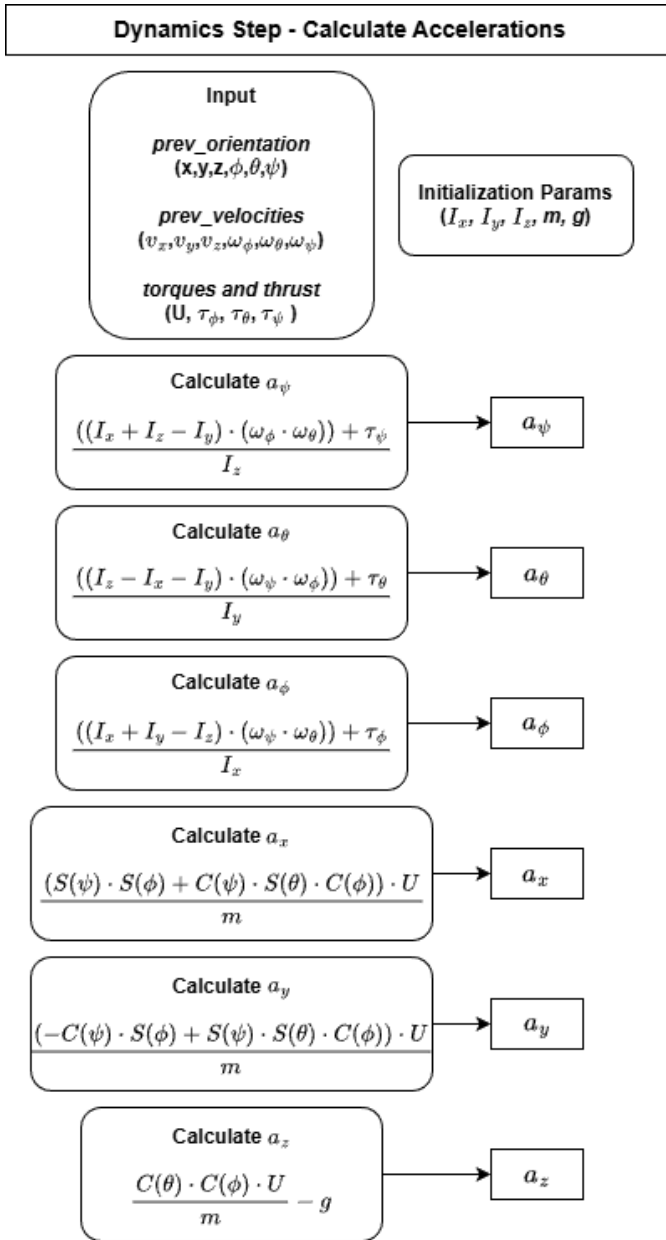


Fig. 6. Dynamics Step - Acceleration Schematic

and rotational states (position, velocity, orientation, angular rates) at each integration timestep.

FR-02 — Six-Channel PID Control The system shall implement six independent PID controllers governing roll, pitch, yaw, and their respective angular rates, each with configurable gains (Kp, Ki, Kd).

FR-03 — PID Gain Parameterization The system shall accept a gain vector of 18 parameters (Kp, Ki, Kd for each of the six controllers) as input to each simulation run, with user-definable upper and lower bounds per parameter.

FR-04 — Monte Carlo Sample Generation The system shall generate N candidate PID gain vectors by sampling from Gaussian distribution within the given bounds on the GPU.

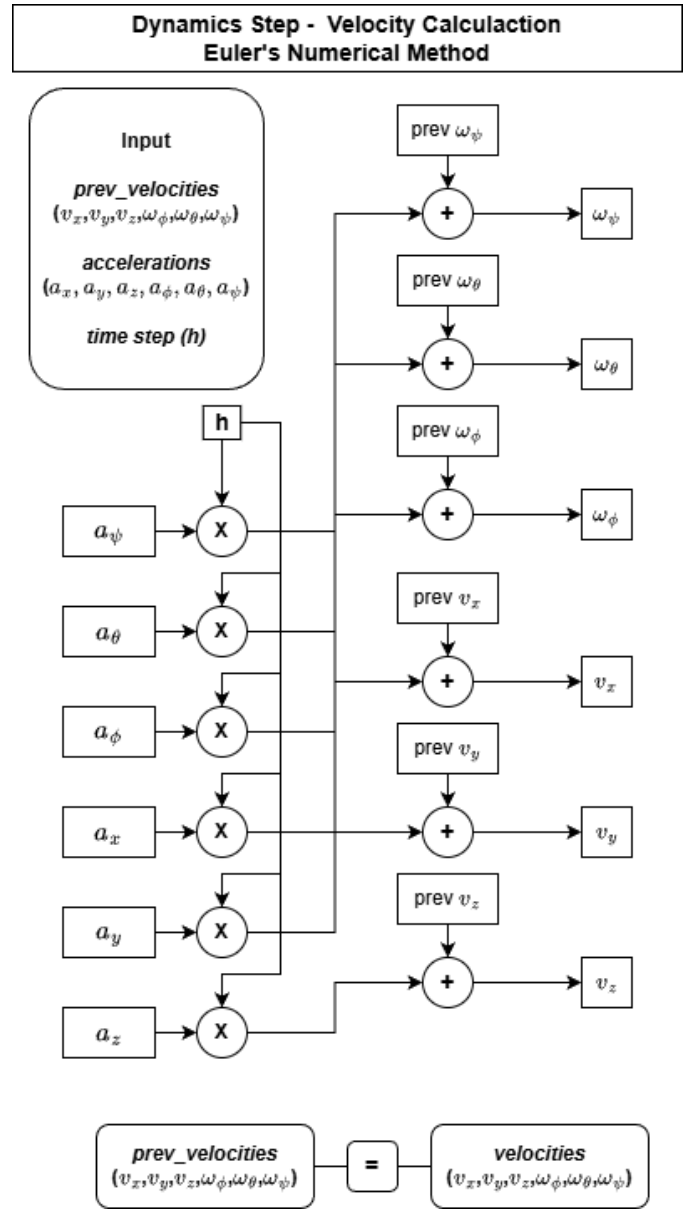


Fig. 7. Dynamics Step - Velocities Schematic

FR-05 — GPU-Parallel Simulation Execution The system shall execute N independent closed-loop simulations in parallel on the Jetson GPU, where each simulation integrates the quadrotor model with a distinct candidate gain vector and a given command trajectory.

FR-06 — Optimal Gain Selection The system shall identify and return the PID gain vector that minimizes the cost function across all N Monte Carlo samples, along with its cost value.

FR-07 — Statistical Summary Output The system shall compute and report summary statistics of the Monte Carlo tests.

FR-08 — Real-Time and Fast Modes The system shall support both real time simulation and full-speed modes. Where in the latter system behavior will be observed from logs.

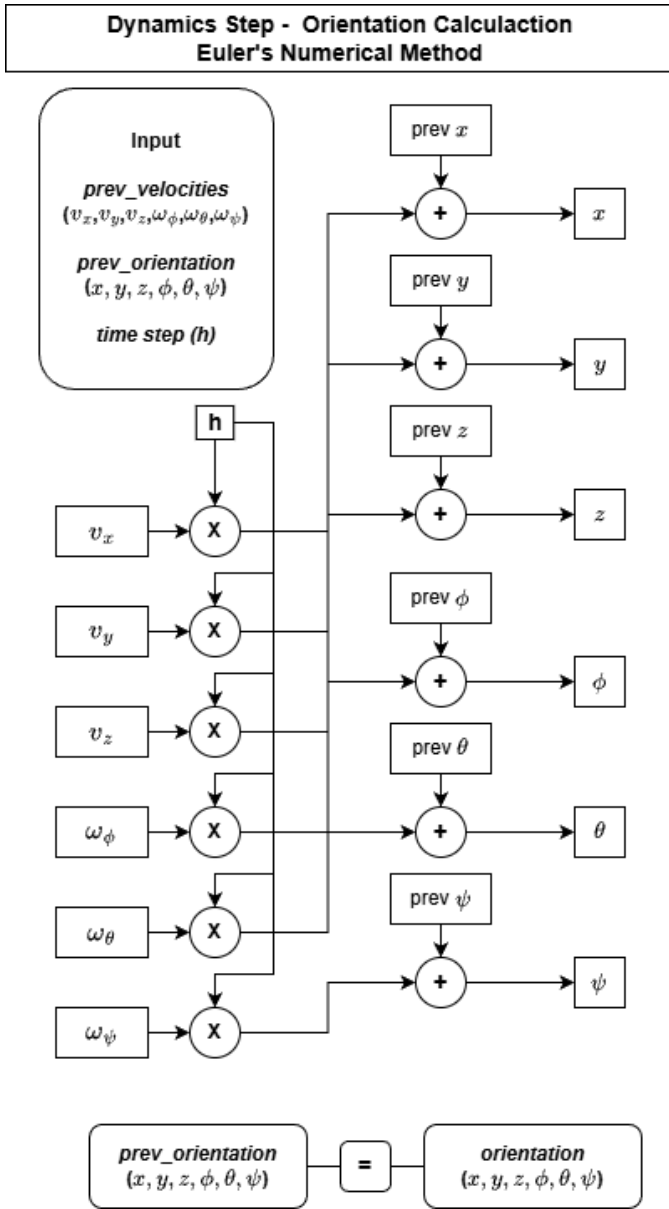


Fig. 8. Dynamics Step - Orientation Schematic

FR-09 — Result Logging and Export The system shall log the full set of evaluated gain vectors and their corresponding costs to persistent storage, in a format suitable for post-processing (CSV or binary).

FR-10 — Simulation Timestep Configuration The system shall allow the user to configure the integration timestep (dt) and the total simulation horizon (T) for each Monte Carlo run.

FR-11 — Task Completion Before Time-step All real-time tasks should finish before the desired time-step in real-time.

2) *Non-Functional Requirements:* NFR-01 — Monte Carlo Throughput The system shall evaluate at least 1,000 complete closed-loop simulations per second on the target Jetson GPU.

NFR-02 — Memory Footprint per Simulation Thread Each GPU thread shall consume no more than 2 KB of local mem-

ory (registers + local variables), excluding shared memory.

NFR-03 — Numerical Stability The quadrotor simulation shall remain numerically stable (no NaN or Inf states) for all PID gain combinations within the user-defined parameter bounds, using a fixed-step integrator with dt.

NFR-04 — Result Determinism Given identical input parameters and randomized parameters, the system shall produce identical results across repeated runs on the same Jetson hardware.

NFR-05 — Latency from Command to Optimized Gains The end-to-end latency from receiving a new command trajectory to outputting the optimized PID gain vector shall not exceed 15 seconds.

NFR-06 — Scalability with Sample Count The system's execution time shall scale linearly with the number of Monte Carlo samples N, for N between 1,000 and 50,000.

NFR-07 — Maintainability and Code Quality The code shall be structured with separation between the dynamic model, PID controller, cost function, and Monte Carlo harness, allowing any component to be modified independently.

D. Timing and Execution Model

The task execution model of our proposed system is both periodic and event-driven. Periodically executing part of the system is real-time simulation of the drone dynamics and PID controller calculations. Event-driven parts start with desired position and rotation info coming from the remote monitor. This new position task starts the GPU driven Monte Carlo evaluation cycle where the system tries to find the best possible PID gain set for this task.

Timing is crucial for such a simulation and testing system as the simulation is expected to run dynamics and control part faster than the actual time step in real time. Also if such an architecture is desired to be used on a real quadrotor as its flight optimizer the Monte Carlo testing should not take too long to finish as the moving task is not executed, or even though it is executed it should not delay the outputs of the optimized controller.

E. Model of Computation

The selected model of computation is Hybrid Automata. The behavior of the system consists of continuous dynamics and discrete computations. Here continuous dynamics includes the continuous change of the drone's orientation, velocities, and accelerations. Discrete computation includes the sample driven calculations, updates occurring at discrete periodic instances and the state transitions happening in the system such as system initialization, dynamics calculation step, control step and logging.

F. Assumptions and Design Limitations

The performance of this PID optimizer and drone simulator CPU-GPU heterogeneous architecture is assumed to be similar on a real drone on ideal environmental factors as the simulated environment does not include harsh conditions as the limited computational power on this embedded device prevents such detailed representations of real world.

Also, limited number of CUDA cores is another obstacle in finding the best possible controller for the given quadrotor in short time. This may result in finding PID gain sets in local minima in order to finish evaluation task in a short time.

G. Resource Model

The Jetson boards share a unified memory architecture — CPU and GPU access the same physical DRAM. This is both an advantage and a constraint. On the advantage side, PCIe transfer is avoided. On the constraint side, quadrotor simulation state, PID controller buffers, and the Monte Carlo sample arrays all compete for the same pool.

Each quadrotor state vector and each controller will need its own space. On a single instance, this is a trivially small size. However, the GPU Monte Carlo side is where memory matters. If you run N parallel simulations, each needing its own copy of the quadrotor state plus the PID gain set being evaluated, memory scales linearly with N . This can result in short evaluation intervals or partitioning the Monte Carlo runs into chunks to fit in available memory.

V. METHODOLOGY

A. Implementation Details (Implementation Track)

The system consists of four main components: a quadrotor dynamics simulator, a cascaded PID controller, a GPU-accelerated Monte Carlo evaluation engine, and a visualization interface. Below we describe each and clearly distinguish between our own implementation and reused libraries.

Quadrotor Dynamics Simulator (custom implementation): The quadrotor is modeled as a rigid body with four rotors, each generating thrust $F = k \cdot \omega^2$. The nonlinear equations of motion compute translational accelerations through rotation matrices (using Euler angles ϕ, θ, ψ) and rotational accelerations through gyroscopic cross-coupling terms and moments of inertia, following the standard formulation also used in [2]. The 12-dimensional state vector contains position, velocity, Euler angles, and angular rates. Integration uses forward Euler at $h = 0.005$ s, with computational complexity considerations as discussed in [3]. This simulator is implemented from scratch in C++.

Cascaded PID Controller (custom implementation): The controller has two layers. The outer loop controls position: `pid_z` adds a thrust correction to the hover feedforward ($m \cdot g$), while `pid_x` and `pid_y` output desired pitch (θ_d) and roll (ϕ_d) angles, clamped to $\pm 25^\circ$. The inner loop controls attitude: `pid_phi`, `pid_theta`, `pid_psi` compute torque commands from angle errors. All six PID controllers use derivative-on-measurement (computing $-d(\text{measured})/dt$ instead of $d(\text{error})/dt$) to avoid derivative kick, and integral anti-windup with configurable limits. The gains are designed from target natural frequencies ensuring all closed-loop poles satisfy the Euler stability condition $|1 + h \cdot s| < 1$. A motor mixer inverts the force/torque allocation to obtain four rotor speeds $\omega_1 \dots \omega_4$, with negative thrust clamped to zero. The entire controller is implemented from scratch in C++.

GPU Monte Carlo Evaluation Engine (to be implemented): This component will use NVIDIA CUDA. The overall approach follows the GPU-accelerated parallel rollout architecture presented in [2], where each GPU thread independently simulates one full trajectory with perturbed parameters. The CPU transfers state, PID gains, system parameters, perturbation sigmas, and waypoints to the GPU via `cudaMemcpy(HostToDevice)`. A custom CUDA kernel launches N threads; each applies stochastic perturbations (sampled via `cuRAND`), runs the same cascaded PID and dynamics logic for T timesteps, and stores the trajectory. Results return via `cudaMemcpy(DeviceToHost)`. The use of Monte Carlo sampling for evaluating quadrotor controllers under uncertainty is also motivated by [1], though our approach uses PID rather than MPC. `cuRAND` (existing NVIDIA library) provides random number generation; the kernel logic is our own implementation.

Visualization Interface (custom, using SFML library): A replay tool implemented using the SFML graphics library (existing open-source library) renders a 3D-perspective drone view, trailing path, HUD with state values, and time-series graphs on a monitoring PC.

Communication and Integration: CPU–GPU transfer uses `cudaMemcpy`. The CPU main loop runs at 200 Hz: each step updates waypoints, computes cascaded PID, applies motor mixer, and advances dynamics. Periodically, state is sent to the GPU for Monte Carlo evaluation; upon return, the PID Gain Selector evaluates robustness statistics and may update gains.

Custom vs. reused: **Custom:** dynamics simulator, cascaded PID, motor mixer, CUDA kernel, PID Gain Selector, visualization, integration. **Reused:** CUDA runtime & `cuRAND` (NVIDIA), SFML (graphics), OpenMP (CPU baseline).

C. Algorithm Description

The GPU Monte Carlo rollout kernel mirrors the CPU-side control and dynamics logic but adds stochastic perturbations, following the single-thread-per-rollout SIMD pattern from [2]. Each of the N threads executes the full pseudocode below independently:

```
KERNEL McRollout(state, par, gains,
                  sig, des, res, N, T):
    tid = threadIdx()
    if tid >= N: return

    s = copy(state)    // local state
    p = copy(par)       // local params

    // Perturb params (once per rollout)
    rng = init_curand(seed, tid)
    p.mass += gauss(rng) * sig.mass
    p.Ix   += gauss(rng) * sig.I
    p.Iy   += gauss(rng) * sig.I
    s.x    += gauss(rng) * sig.pos
    s.y    += gauss(rng) * sig.pos
    s.z    += gauss(rng) * sig.pos

    pids = init_pids(gains) // 6 PIDs

    for t = 0 to T-1:
```

```

// --- Outer PID (position) ---
U = p.mass*g
  + pid(pz, des.z, s.z, h)
td = pid(px, des.x, s.x, h)
pd = -pid(py, des.y, s.y, h)
clamp(td, -0.44, 0.44)
clamp(pd, -0.44, 0.44)

// --- Inner PID (attitude) ---
tp = pid(pphi, pd, s.phi, h)
tt = pid(ptheta, td, s.theta, h)
ts = pid(ppsi, des.psi, s.psi, h)

// --- Motor mixer ---
f1 = U/4 + ts/(4L) + tt/(2L)
f2 = U/4 - ts/(4L) + tp/(2L)
f3 = U/4 + ts/(4L) - tt/(2L)
f4 = U/4 - ts/(4L) - tp/(2L)
f1..f4 = max(0, f1..f4)
w1..w4 = sqrt(f1..f4 / k)

// --- Per-step control noise ---
w1..w4 += gauss(rng)*sig.mot

// --- Forward Euler step ---
compute forces from w1..w4
compute accel (rotation matrix)
s.vel += h * accel
s.pos += h * s.vel
compute ang_accel (inertia)
s.wrate += h * ang_accel
s.angle += h * s.wrate

res[tid*T + t] = s

miss[tid] = dist(s.pos, des.pos)

```

The CPU-side main loop:

```

MAIN LOOP (200 Hz, CPU):
for each step i:
  des = waypoint(i)
  ctrl.update(drone, des, h)
  drone.dynamics_step()

if i % eval_period == 0:
  gpu_send(state, par, gains,
            sig, des)
  launch McRollout(N threads)
  res = gpu_receive()
  stats = compute_stats(res)
  if stats.success < threshold:
    gains = select_best(cands, res)
    ctrl.set_gains(gains)

```

The motor mixer equations directly invert the force-to-torque mapping of the drone’s rotor configuration. The `compute()` function implements: $u = K_p \cdot e + K_i \cdot \int e \, dt + K_d \cdot (-d(\text{meas})/dt)$, where the derivative term uses the measured value rather than the error to prevent setpoint-change spikes. These are consistent with the controller structure in our CPU-side C++ implementation.

A. Platform and Tools

The system will run on an NVIDIA Jetson board that provides both ARM CPU cores and a CUDA-capable GPU on a single platform. The specific Jetson model has not been

decided yet. A desktop machine with a discrete NVIDIA GPU will also be used for performance comparison. The operating system will be JetPack SDK (Ubuntu-based) on the Jetson.

The software is being developed in C++ with CUDA. The quadrotor simulator is a custom implementation with a 12-dimensional state vector, nonlinear equations of motion, and forward Euler integration at $h = 0.005$ s, taking into account real-time computational constraints as discussed in [3]. The controller uses a cascaded PID architecture with an outer position loop and an inner attitude loop. The CPU will run the nominal control loop at 200 Hz, while the GPU will perform Monte Carlo robustness evaluation by running N parallel rollouts with perturbed parameters using custom CUDA kernels, following the GPU-accelerated trajectory evaluation approach presented in [2]. An SFML-based visualization tool will be used for monitoring and replay on a connected PC.

B. Datasets / Workloads

The system is entirely simulation-based. The nominal quadrotor parameters are: $m = 1.4$ kg, $L = 1.0$ m, $I_x = I_y = 0.0116$ kg·m², $I_z = 0.0232$ kg·m², $k = 1 \times 10^{-6}$, $g = 9.8$ m/s².

Two waypoint scenarios are currently implemented: a vertical climb to $z = 20$ m ($t = 0$ –10 s), and a lateral maneuver to $x = 3$ m, $y = 2$ m while maintaining altitude ($t = 10$ –20 s). Additional scenarios will be added for future applications such as path planning and charging station routing.

Monte Carlo rollouts will be subject to stochastic perturbations including initial condition deviations, mass and inertia uncertainties, control input noise, and wind disturbances, similar to the perturbation classes defined in [2]. The use of Monte Carlo sampling for quadrotor controller evaluation under uncertainty has also been explored in [1] with a model predictive control approach. The perturbation magnitudes and the ranges for rollout count N and simulation horizon T will be determined during experimentation.

C. Evaluation Metrics

We plan to compare three execution modes: (1) **CPU-Serial** (single ARM core), (2) **CPU-Parallel** (OpenMP, multiple ARM threads), and (3) **GPU-CUDA** (parallel CUDA kernel).

The planned computational metrics are: **latency** (wall-clock time to complete all rollouts), **throughput** (rollouts per second), **speedup** (CPU-Serial time / GPU time over varying N , following the benchmarking methodology in [2]), and **energy consumption** (measured via `tegrastats` on the Jetson, CPU-only vs. GPU-accelerated in mJ).

The planned control robustness metrics are: **miss distance** (Euclidean distance from final state to target), **settling time**, **overshoot**, and **success rate** (fraction of rollouts reaching the target within tolerance). Sensitivity analysis will identify which perturbation parameter affects PID performance the most, as demonstrated in [2] for mass, inertia, and control input perturbations on a quadcopter system. Unlike [1], which applies Monte Carlo methods within a model predictive control framework, our approach focuses on evaluating a cascaded PID controller on an embedded GPU platform. The PID Gain

Selector will evaluate results across different gain combinations and update the controller accordingly.

VI. SYSTEM-LEVEL CONSIDERATIONS

Real-time constraints and latency: The CPU-side control loop must run at 200Hz (5 ms period) without interruption to maintain drone stability. The GPU Monte Carlo evaluation runs asynchronously and must not block the control loop. The latency of `cudaMemcpy` transfers on the Jetson’s shared memory architecture will be a key factor.

Energy efficiency: Power consumption of CPU-only versus GPU-accelerated execution is an important consideration for embedded platforms and will be evaluated.

Scalability: The system should scale with the number of parallel rollouts N . On desktop GPUs, [2]demonstrated strong scaling up to thousands of rollouts. On the Jetson’s smaller GPU, the crossover point where GPU becomes faster than CPU is expected to differ. Characterizing this on embedded hardware is one of the main goals of the project.

Reliability and fault tolerance: If the GPU evaluation fails or times out, the CPU control loop must continue with the current PID gains without interruption. GPU results are advisory — they can update gains but their absence will not cause loss of control.

Security considerations: The system operates as a self-contained simulation on the Jetson with no external interfaces during operation. Security risks are minimal at this stage.

All items above are identified considerations. Detailed measurement and analysis will be presented in the final report.

ACKNOWLEDGMENTS

Through this project, we hope to gain hands-on experience in GPU programming with CUDA, embedded system development on the Jetson platform, quadrotor dynamics modeling, and PID controller design. We also expect to develop a deeper understanding of Monte Carlo methods for robustness evaluation and CPU-GPU workload partitioning in resource-constrained environments.

REFERENCES

- [1] K. Masuda and K. Uchiyama, “Attitude control system for quadrotor using robust Monte Carlo model predictive control,” *Actuators*, vol. 13, no. 11, p. 443, 2024.
- [2] R. Melzer, M. Gandhi, R. Schlossman, K. Williams, and J. Parish, “CUDA for rapid controller robustness evaluation: A tutorial,” *Sandia National Laboratories Technical Report*, 2021.
- [3] G. Delgado-Reyes, J. S. Valdez-Martínez, M. Hernández-Pérez, K. R. Pérez-Daniel, and P. J. García-Ramírez, “Quadrotor real-time simulation: A temporary computational complexity-based approach,” *Mathematics*, vol. 10, no. 12, p. 2032, 2022.
- [4] V. M. Babu, K. Das, and S. Kumar, “Designing of self tuning PID controller for AR drone quadrotor,” *2017 18th International Conference on Advanced Robotics*, pp. 167–172, 2017.
- [5] M. Rizzello and M. Molaei, “Motor-quadrotor simulations based on FOC and cuckoo PID tuning,” *IEEE International Conference*, 2024.
- [6] A. Iyer, “Modelling, simulation, and implementation of PID controller on quadrotors,” *ResearchGate Preprint*, 2021.
- [7] A. T. Reenberg, J. K. Huusom, and J. B. Jorgensen, “A high-performance Monte Carlo simulation toolbox for uncertainty quantification of closed-loop systems,” *2022 IEEE Conference on Decision and Control*, 2022.

- [8] Unknown, “Dynamic response optimization for quadrotor UAVs using a parameter self-tuning fuzzy PID control method,” *IEEE*, 2024.
- [9] O. Doukhi and D. J. Lee, “Intelligent controller design for quad-rotor stabilization in presence of parameter variations,” *Journal of Advanced Transportation*, vol. 2017, p. 4683912, 2017.
- [10] G. M. Chari, A. G. Kamath, P. Elango, and B. Acikmese, “Fast Monte Carlo analysis for 6-DoF powered-descent guidance via GPU-accelerated sequential convex programming,” *AIAA SciTech Forum*, 2024.
- [11] NVIDIA, “Getting started with Jetson Nano 2GB developer kit,” *NVIDIA Developer Documentation*, 2024.